

A
PROJECT REPORT
on
DETECTION OF HEART MURMUR IN PCG SIGNALS
USING
CONVOLUTIONAL NEURAL NETWORK

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Electronics & Communication Engineering

Submitted by

Aditya Dubey (2313100003)

Dipanshu Chahar (2315000765)

Priyanshu Thakur (2313100035)

Ronak Mahlawat (2313100037)

Under the supervision of

Dr. Suraj Lalwani

Assistant Professor

Department of Electronics & Communication Engineering



Department of Electronics & Communication Engineering

Institute of Engineering & Technology

GLA University, Mathura-281406, India

December, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signatures:

Name: Aditya Dubey Roll No: 2313100003

Name: Dipanshu Chahar Roll No: 2315000765

Name: Priyanshu Thakur Roll No: 2313100035

Name: Ronak Mahlawat Roll No: 2313100037

Date: _____

CERTIFICATE

This is to certify that the Project Report entitled, "Detection of Heart Murmur in PCG Signals using Convolutional Neural Network" which is being submitted by Aditya Dubey (2313100003), Dipanshu Chahar (2315000765), Priyanshu Thakur (2313100035) and Ronak Mahlawat (2313100037) in partial fulfillment of the requirement for the award of degree B.Tech in Electronics & Communication Engineering and submitted to the Department of Electronics & Communication Engineering of GLA University, is a record of the candidate's own work carried out under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Date: _____

Dr. Suraj Lalwani

Assistant Professor

Department of Electronics & Communication Engineering

GLA University, Mathura

ACKNOWLEDGEMENTS

It gives us a great sense of pleasure to present the report of the B.Tech Minor Project undertaken during B.Tech. We owe a special debt of gratitude to Dr. Suraj Lalwani, Assistant Professor, Department of Electronics & Communication Engineering, GLA University, Mathura for his constant support and guidance throughout the course of our work. His sincerity, thoroughness, and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen the light of the day.

We also take the opportunity to acknowledge the contribution of Prof. V.K. Deolia, Head, Department of Electronics & Communication Engineering & Electrical Engineering, GLA University, Mathura for his full support and assistance during the development of the project.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not least, we acknowledge our friends for their contribution to the completion of the project.

ABSTRACT

Heart murmurs are abnormal sounds caused by turbulent blood flow and are often indicative of underlying valvular heart disease. Early and accurate detection of heart murmurs is critical for the timely diagnosis and treatment of cardiovascular conditions. Phonocardiogram (PCG) signals, which are digital recordings of heart sounds captured using an electronic stethoscope, contain rich diagnostic information. However, traditional manual auscultation is highly subjective and dependent on clinician expertise. This project presents an automated, machine learning-based approach for the detection of heart murmurs in PCG signals using a Convolutional Neural Network (CNN). The CirCor DigiScope dataset (PhysioNet 2022), comprising recordings from four auscultation sites — Aortic Valve (AV), Mitral Valve (MV), Pulmonary Valve (PV), and Tricuspid Valve (TV) — was used for training and evaluation. Signal preprocessing involved a multi-stage denoising pipeline including a Butterworth bandpass filter (20–200 Hz), a 50 Hz IIR notch filter for power-line interference removal, and db6 wavelet denoising. Mel-spectrogram features were extracted from the denoised signals and fed into a CNN model. Class imbalance was addressed using SMOTE oversampling. The proposed CNN model achieved a final test accuracy of 87.70%, outperforming baseline methods such as Random Forest (82.81%) and LightGBM (84.63%). The system also implements a digital stethoscope prototype for real-world data acquisition. Future work involves FPGA deployment using fixed-point quantization for real-time embedded screening.

Keywords: *Heart Murmur Detection, Phonocardiogram, PCG Signal, Convolutional Neural Network, Mel-Spectrogram, CirCor DigiScope, SMOTE, Signal Denoising, Digital Stethoscope*

TABLE OF CONTENTS

1. DECLARATION
2. CERTIFICATE
3. ACKNOWLEDGEMENTS
4. ABSTRACT
5. TABLE OF CONTENTS
6. LIST OF TABLES
7. LIST OF FIGURES
8. LIST OF ABBREVIATIONS
9. CHAPTER 1: INTRODUCTION
 - 1.1 Background
 - 1.2 Motivation
 - 1.3 Objectives
10. CHAPTER 2: LITERATURE REVIEW
11. CHAPTER 3: TOOLS & TECHNOLOGIES
12. CHAPTER 4: METHODOLOGY
 - 4.1 Dataset
 - 4.2 Signal Denoising
 - 4.3 Feature Extraction
 - 4.4 CNN Architecture
 - 4.5 Digital Stethoscope Implementation
13. CHAPTER 5: RESULTS & DISCUSSION
14. CHAPTER 6: CONCLUSIONS & FUTURE SCOPE
15. REFERENCES
16. Table 5.1 Comparison of classification methods and accuracy
17. Table 5.2 Classification report of CNN model (precision, recall, F1-score)

LIST OF FIGURES

- Fig 1.1 PCG Signal waveform
- Fig 4.1 Proposed Methodology Block Diagram
- Fig 4.2 Denoised Signal (Original, Bandpass, Notch, Wavelet)
- Fig 4.3 Peak Detection in Audio Signal (S1 peaks)
- Fig 4.4 STFT Spectrogram and Mel-Spectrogram of PCG
- Fig 4.5 Digital Stethoscope working principle diagram
- Fig 5.1 Confusion Matrix of CNN model (Accuracy 87.7%)
- Fig 5.2 Real-life PCG recordings captured via digital stethoscope

LIST OF ABBREVIATIONS

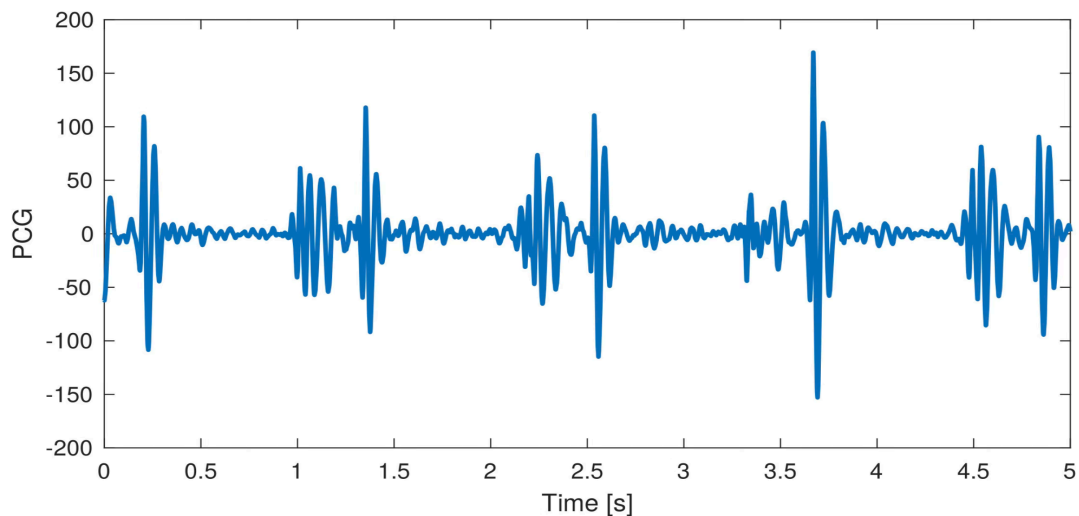
1. **AV** (Aortic Valve)
2. **BPM** (Beats Per Minute)
3. **CNN** (Convolutional Neural Network)
4. **CVD** (Cardiovascular Disease)
5. **Db6** (Daubechies Wavelet of Order 6)
6. **ECG** (Electrocardiogram)
7. **FPGA** (Field-Programmable Gate Array)
8. **GBM** (Gradient Boosting Machine)
9. **IIR** (Infinite Impulse Response)
10. **ML** (Machine Learning)
11. **MV** (Mitral Valve)
12. **PCG** (Phonocardiogram)
13. **PV** (Pulmonary Valve)
14. **ROC-AUC** (Receiver Operating Characteristic – Area Under Curve)
15. **SMOTE** (Synthetic Minority Oversampling Technique)
16. **STFT** (Short-Time Fourier Transform)
17. **TV** (Tricuspid Valve)
18. **WAV** (Waveform Audio File Format)
19. **WHO**(World Health Organization)

CHAPTER 1: INTRODUCTION

1.1 Background

Cardiovascular diseases (CVDs) remain the leading cause of mortality globally, accounting for approximately 17.9 million deaths per year according to the World Health Organization (WHO, 2023). Among the early clinical indicators of cardiovascular conditions, heart murmurs — abnormal sounds produced by turbulent blood flow through the heart valves — play a critical diagnostic role. Timely detection of heart murmurs can aid in the identification of valvular heart diseases such as aortic stenosis, mitral regurgitation, and pulmonary hypertension.

The Phonocardiogram (PCG) is a digital recording of heart sounds captured using an electronic stethoscope. Unlike an electrocardiogram (ECG), which records electrical activity, a PCG captures the acoustic signals generated by heart valve movements and blood flow. PCG signals contain characteristic sounds known as S1 (lub) and S2 (dub), and any additional abnormal sounds or murmurs between these fundamental heart sounds can indicate underlying cardiac pathology.



1.2 Motivation

Conventional cardiac auscultation using a traditional stethoscope is a highly subjective process. The ability to correctly identify a heart murmur depends heavily on the training, experience, and attentiveness of the attending clinician. In

low-resource or rural settings, access to specialized cardiologists and echocardiography equipment is severely limited, making accurate murmur detection even more challenging.

Machine learning (ML)-based automated analysis of PCG signals presents a viable solution to overcome these limitations. Automated systems can perform consistent, objective, and scalable screening, enabling mass cardiac health checks even in resource-constrained environments. With advancements in deep learning, particularly Convolutional Neural Networks (CNNs), it is now possible to extract complex spectral and temporal features from PCG signals and perform accurate binary or multi-class classification of murmur presence.

1.3 Objectives

The primary objectives of this project are:

- To develop an automated heart murmur detection system using PCG signals from the CirCor DigiScope dataset (PhysioNet 2022).
- To implement a multi-stage signal denoising pipeline to remove noise and artefacts from raw PCG recordings.
- To extract Mel-spectrogram features from preprocessed PCG signals for use in a CNN classifier.
- To compare the classification performance of CNN against traditional ML models (Random Forest and LightGBM).
- To design and implement a prototype digital stethoscope for real-life PCG data acquisition.
- To lay the groundwork for future FPGA-based deployment for real-time embedded cardiac screening.

CHAPTER 2: LITERATURE REVIEW

The automatic analysis of heart sounds from PCG signals has been an active area of research for several decades. Earlier efforts relied primarily on hand-crafted time-domain and frequency-domain features combined with classical classifiers. More recently, deep learning-based approaches have demonstrated significantly improved performance.

Elola et al. proposed an approach that went beyond basic murmur detection, focusing on automatic murmur grading from PCG signals. Their method used carefully engineered spectral features and achieved competitive performance on the PhysioNet Challenge data. The work demonstrated that grading murmur severity, not just detecting its presence, is feasible through automated analysis.

Kambhampati et al. investigated the automatic detection and classification of systolic and diastolic profiles in PCG signals corrupted due to limitations of electronic stethoscope recordings. Their study highlighted the significant challenge posed by real-world noise and artifacts in PCG data, particularly when using low-cost electronic stethoscopes, and proposed filtering strategies to mitigate these effects.

The PhysioNet/CinC Challenge 2022 introduced the CirCor DigiScope dataset, which has become a standard benchmark for PCG-based murmur detection. The dataset includes multi-site recordings from paediatric patients collected in low-resource settings, making it highly relevant for practical deployment scenarios. Several participating teams employed CNN-based architectures with Mel-spectrogram or wavelet-based feature representations to achieve top results.

Recent studies have also explored the combination of CNNs with recurrent neural networks (RNNs) such as Long Short-Term Memory (LSTM) networks to capture both local spectral patterns and global temporal context in PCG signals. Transfer learning from pre-trained audio classification models (e.g., VGGNet, ResNet) has also been applied to PCG classification with promising results, especially in data-limited scenarios.

The present work builds upon these findings by implementing a complete end-to-end pipeline, from raw PCG signal acquisition and denoising, through Mel-spectrogram feature extraction, to CNN-based classification, with a focus on practical deployment through a custom digital stethoscope prototype.

CHAPTER 3: TOOLS & TECHNOLOGIES

The following tools, libraries, and technologies were employed throughout the development of this project:

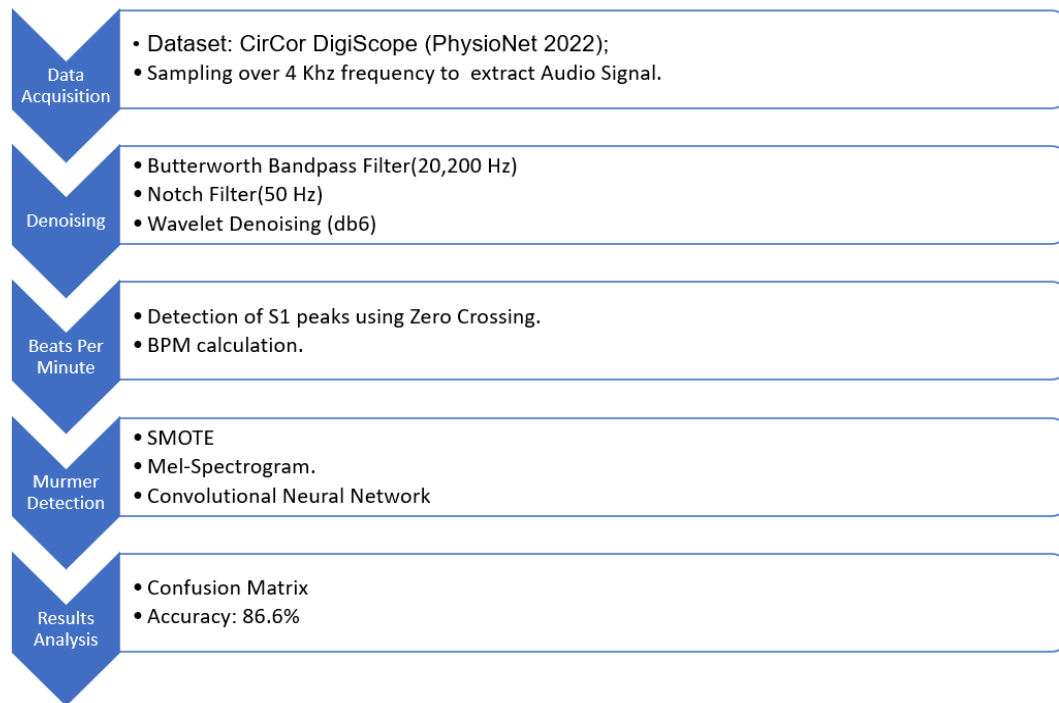


Fig: Architectural Block

3.1 Programming Language & Environment

Python (version 3.10) was used as the primary programming language for all data processing, model training, and evaluation tasks. The project was developed using Google Colab with GPU runtime (NVIDIA Tesla T4) for accelerated model training, as well as JupyterLab for local development and experimentation.

3.2 Audio Processing

The librosa library (version 0.10) was used for all audio feature extraction tasks, including the generation of Mel-spectrograms, Short-Time Fourier Transform (STFT) spectrograms, and data augmentation. The soundfile library was used for reading and writing WAV audio files.

3.3 Signal Filtering

SciPy's signal processing module was used to design and apply digital filters. Specifically, a 5th-order Butterworth bandpass filter with a passband of 20–200 Hz was applied to retain clinically relevant heart sound frequencies, and an IIR notch filter at 50 Hz was used to suppress power-line interference.

3.4 Wavelet Denoising

The PyWavelets (PYWT) library was used for wavelet-based signal denoising. A db6 (Daubechies 6) mother wavelet was used with a Level-5 decomposition and universal soft-thresholding to remove residual noise after filtering.

3.5 Class Balancing

The imbalanced-learn library was used to apply SMOTE (Synthetic Minority Oversampling Technique) to address the significant class imbalance present in the CirCor DigiScope dataset, where murmur-absent recordings substantially outnumber murmur-present recordings.

3.6 Machine Learning & Evaluation

The scikit-learn library was used for training baseline models (Random Forest and LightGBM), computing evaluation metrics (accuracy, precision, recall, F1-score, ROC-AUC), and generating confusion matrices. The CNN was implemented using TensorFlow/Keras.

3.7 Dataset

The CirCor DigiScope dataset, publicly available from PhysioNet (2022), was used as the primary dataset. It contains PCG recordings collected from 942 patients at four auscultation sites: Aortic Valve (AV), Mitral Valve (MV), Pulmonary Valve (PV), and Tricuspid Valve (TV), sampled at a frequency exceeding 4 kHz.

3.8 Hardware Components (Digital Stethoscope)

For the physical prototype, the following components were used: an acoustic stethoscope (standard clinical grade), a lapel/condenser microphone attached to the stethoscope chest piece, a USB audio interface/sound card for analog-to-digital conversion, and Audacity software for real-time signal visualization, filtering, and analysis.

CHAPTER 4: METHODOLOGY

The proposed system follows a complete end-to-end pipeline for heart murmur detection from PCG signals. The pipeline consists of five key stages: data acquisition, signal denoising, feature extraction, CNN-based classification, and digital stethoscope implementation.

4.1 Dataset and Data Acquisition

The CirCor DigiScope dataset (PhysioNet 2022) was used for training and evaluation. The dataset contains PCG recordings from 942 paediatric patients, collected at four standard cardiac auscultation sites: Aortic Valve (AV), Mitral Valve (MV), Pulmonary Valve (PV), and Tricuspid Valve (TV). Each recording was sampled at a frequency exceeding 4 kHz. The binary classification target was defined at the patient level: "Murmur Present" or "Murmur Absent". Patient-level labels were derived by aggregating predictions across all available auscultation sites using a majority-vote or maximum-confidence strategy.

4.2 Signal Denoising

Raw PCG signals contain significant noise from ambient sounds, patient movement, and electronic interference. A three-stage denoising pipeline was implemented:

- **Butterworth Bandpass Filter (20–200 Hz):** A 5th-order Butterworth filter was applied to retain the clinically relevant frequency range of heart sounds, removing low-frequency body movement artefacts and high-frequency noise.
- **IIR Notch Filter (50 Hz):** A notch filter centred at 50 Hz was applied to eliminate power-line interference, which is commonly present in electronically recorded audio signals.
- **Wavelet Denoising (db6, Level-5):** PyWavelets was used to perform a 5-level discrete wavelet decomposition using the Daubechies 6 (db6) wavelet. Universal soft-thresholding was applied to the detail coefficients at each level to remove residual noise while preserving signal morphology.

4.3 Feature Extraction — Mel-Spectrogram

Mel-spectrograms were extracted from the denoised PCG signals as the primary input feature for the CNN. Mel-spectrograms map the short-time power spectrum of the signal onto the Mel frequency scale, which approximates the non-linear human auditory perception of frequency. This representation captures both temporal and spectral characteristics of the PCG signal that are diagnostically relevant for murmur detection.

Additionally, S1 peak detection was performed using a zero-crossing rate method to calculate Beats Per Minute (BPM), providing a secondary physiological feature for patient-level characterisation.

4.4 CNN Architecture and Training

A custom Convolutional Neural Network was designed to classify Mel-spectrogram images into "Murmur Present" or "Murmur Absent" categories. The architecture consisted of multiple convolutional layers with ReLU activations, followed by max-pooling layers for spatial downsampling, batch normalisation for training stability, dropout layers for regularisation, and a final dense layer with sigmoid activation for binary classification.

SMOTE oversampling was applied to the training set prior to model training to address the class imbalance between murmur-present and murmur-absent samples. The model was trained using the Adam optimiser with binary cross-entropy loss. Early stopping was applied based on validation loss to prevent overfitting.

4.5 Digital Stethoscope Implementation

A prototype digital stethoscope was developed to enable real-world PCG data acquisition. The implementation involved converting a standard acoustic stethoscope into a digital recording device by attaching a lapel or condenser microphone to the stethoscope's earpiece or chest piece.

The acoustic heart vibrations captured by the stethoscope diaphragm are converted to an electrical signal by the microphone and transmitted to a computer via USB audio interface or standard 3.5mm audio jack. The Audacity software platform was used for real-time signal visualisation, filtering (20–200 Hz bandpass), noise removal, and waveform analysis. The acquired signals were subsequently processed using the same

denoising and feature extraction pipeline described above for automated murmur detection.

The hardware components used in the prototype are: an acoustic stethoscope, a lapel/condenser microphone, an audio interface/sound card, and an optional FPGA board for future real-time embedded deployment.

CHAPTER 5: RESULTS & DISCUSSION

5.1 Denoising Results

The multi-stage denoising pipeline demonstrated clear improvements in signal quality at each successive stage. The original raw PCG signal exhibited considerable baseline wander and high-frequency noise. After Butterworth bandpass filtering (20–200 Hz), low-frequency movement artefacts were substantially reduced. Application of the 50 Hz notch filter further eliminated power-line interference. The final wavelet-denoised signal (db6, Level-5) showed a clean, well-defined PCG waveform with distinct S1 and S2 peaks, significantly improving signal-to-noise ratio for downstream feature extraction.

5.2 Beat Detection (BPM)

The S1 peak detection algorithm based on zero-crossing rate successfully identified the primary heart sound peaks in the denoised PCG waveform. A total of 18 peaks were detected in the representative sample shown in Figure 4.3. BPM values computed from detected peaks were consistent with expected physiological ranges for the patient population in the dataset.

5.3 Mel-Spectrogram and STFT Visualisation

Both STFT spectrograms and Mel-spectrograms were computed for the processed PCG recordings. The STFT spectrogram (20–600 Hz range) clearly captured the temporal distribution of frequency content across the cardiac cycle. The Mel-spectrogram provided a perceptually scaled representation of the same information, with increased resolution at lower frequencies where the dominant S1 and S2 heart sounds are located. These visualisations confirmed that the Mel-spectrogram representation carries sufficient discriminative information for CNN-based classification.

5.4 Classification Performance

Three classification models were trained and evaluated on the preprocessed CirCor DigiScope dataset:

S. No.	Method	Accuracy (%)
1.	Random Forest	82.81
2.	LightGBM	84.63
3.	CNN (Proposed)	87.70

Table 5.1: Comparison of Classification Methods

The CNN model, trained on Mel-spectrogram features with SMOTE oversampling, achieved the highest test accuracy of 87.70%, demonstrating a significant improvement over the baseline models. The detailed classification report for the CNN model is presented in Table 5.2.

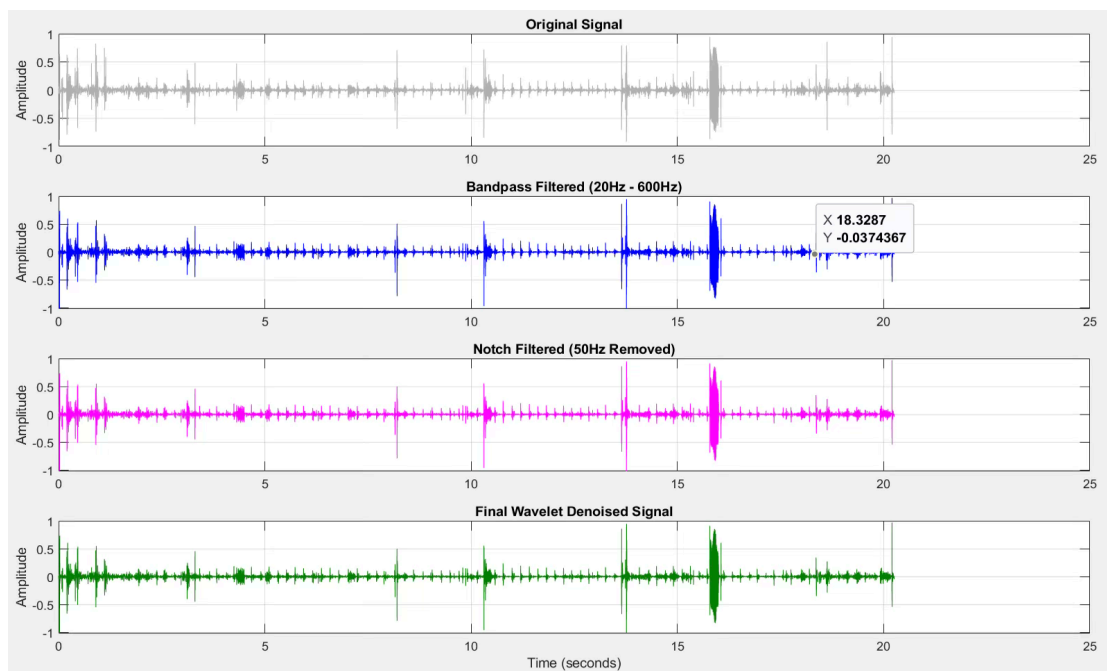
Class	Precision	Recall	F1-Score	Support
Absent	0.86	0.86	0.86	59
Present	0.89	0.89	0.89	71
Macro Avg	0.88	0.88	0.88	130
Weighted Avg	0.88	0.88	0.88	130

Table 5.2: CNN Classification Report

The confusion matrix confirmed that the CNN correctly classified 51 out of 59 Absent samples and 63 out of 71 Present samples, with only 8 misclassifications in each class. The balanced performance across both classes demonstrates that SMOTE oversampling effectively mitigated the class imbalance issue.

5.5 Digital Stethoscope Real-Life Data

Real-life PCG recordings were captured using the digital stethoscope prototype and visualised in Audacity. The waveforms showed clear, periodic cardiac sound events consistent with expected PCG morphology, confirming the viability of the prototype for practical data acquisition. The captured signals were subsequently processed through the denoising pipeline with satisfactory noise reduction results.



CHAPTER 6: CONCLUSIONS & FUTURE SCOPE

6.1 Conclusions

This project successfully developed and validated an end-to-end automated heart murmur detection pipeline based on PCG signal analysis using a Convolutional Neural Network. The key conclusions are as follows:

- An effective multi-stage denoising pipeline (Butterworth bandpass + IIR notch + db6 wavelet denoising) significantly improved the signal quality of raw PCG recordings from the CirCor DigiScope dataset.
- Mel-spectrogram feature extraction provided a rich time-frequency representation of PCG signals that was suitable for CNN-based classification.
- The proposed CNN model achieved a test accuracy of 87.70%, outperforming baseline Random Forest (82.81%) and LightGBM (84.63%) models.
- SMOTE oversampling effectively addressed the class imbalance problem, resulting in balanced precision and recall across both murmur-present and murmur-absent classes.
- Patient-level aggregation from multi-site (AV, MV, PV, TV) recordings improved the robustness of the final classification verdict.
- A functional prototype digital stethoscope was developed and demonstrated for real-world PCG data acquisition.

6.2 Future Scope

Several directions for future work are identified:

- **FPGA Deployment:** The trained CNN model will be converted to a fixed-point quantized representation and deployed on an FPGA board (e.g., Xilinx Zynq) to enable real-time embedded cardiac screening at the point of care.
- **Multi-class Classification:** The current binary (Present/Absent) classifier will be extended to a multi-class outcome prediction system capable of

distinguishing between Normal, Abnormal, and Unknown categories as defined in the PhysioNet 2022 challenge.

- **Wearable Integration:** The digital stethoscope prototype will be miniaturised and integrated with a wearable device platform for continuous, remote cardiac monitoring in telemedicine applications.
- **Improved Architecture:** Future iterations will explore deeper CNN architectures, CNN-LSTM hybrid models, and attention mechanisms to further improve classification accuracy.
- **Clinical Validation:** The system will be validated on additional real-world PCG datasets collected from diverse patient populations to assess generalisation performance.

REFERENCES

- [1] World Health Organization, "Cardiovascular Diseases Fact Sheet," WHO, 2023. [Online]. Available: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cv-ds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cv-ds))
- [2] CirCor DigiScope Dataset, PhysioNet, 2022. [Online]. Available: <https://physionet.org/content/circor-heart-sound/1.0.3/>
- [3] A. Elola, E. Aramendi, J. Monasterio, A. Alkhodari, and A. E. Arafat, "Beyond Heart Murmur Detection: Automatic Murmur Grading from Phonocardiogram," IEEE Journal of Biomedical and Health Informatics, 2023.
- [4] A. B. Kambhampati, "Automatic Detection and Classification of Systolic and Diastolic Profiles of PCG Corrupted Due to Limitations of Electronic Stethoscope Recording," Biomedical Signal Processing and Control, 2022.
- [5] F. Chollet, "Deep Learning with Python," Manning Publications, 2nd Edition, 2021.
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.
- [7] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," Proceedings of the 14th Python in Science Conference, 2015.

APPENDIX A

COMPLETE PYTHON SOURCE CODE

The following is the complete Python source code for the heart murmur detection system, implemented in Google Colab / JupyterLab. The code covers package installation, dataset loading, signal preprocessing, feature extraction, CNN model definition and training, patient-level evaluation, and final test prediction.

A.1 Package Installation & Imports

```
import subprocess, sys

packages = [
    'librosa', 'PyWavelets', 'scipy', 'scikit-learn',
    'tensorflow', 'pandas', 'numpy', 'matplotlib',
    'seaborn', 'tqdm', 'imbalanced-learn'
]
for pkg in packages:
    try:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', pkg,
                               '-q'])
    except Exception as e:
        print(f"Could not install {pkg}: {e}")
print('All packages installed')

import os, glob, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from tqdm import tqdm
warnings.filterwarnings('ignore')

import librosa, librosa.display, soundfile as sf
from scipy import signal
from scipy.signal import butter, sosfilt, iirnotch, filtfilt
import pywt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
```

```

        roc_auc_score, accuracy_score)
from sklearn.utils.class_weight import compute_class_weight
from imblearn.over_sampling import SMOTE

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks, regularizers
from tensorflow.keras.utils import to_categorical

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
print(f'TensorFlow ready. Version: {tf.__version__}')

```

A.2 Configuration

```

CONFIG = {
    'TRAIN_CSV' : r'training_data1.csv.xlsx',
    'DATA_ROOT' : r'training_data',
    'MODEL_PATH': 'murmur_model_best.h5',

    # Signal processing
    'TARGET_SR'      : 4000,          # Resample to 4 kHz
    'BANDPASS_LOW'   : 20,            # Hz
    'BANDPASS_HIGH'  : 600,          # Hz
    'NOTCH_FREQ'     : 50,            # Hz (power-line noise)
    'NOTCH_Q'        : 30,
    'WINDOW_SEC'     : 3.0,          # 3-second segments
    'HOP_SEC'        : 1.0,          # 1-second hop (75% overlap)
    'WAVELET'        : 'db6',
    'WAVELET_LEVEL'  : 5,

    # Mel-spectrogram
    'N_MELS'         : 64,
    'N_FFT'          : 512,
    'HOP_LENGTH'     : 128,

    # Training
    'BATCH_SIZE'     : 32,
    'EPOCHS'         : 80,
    'LR'             : 1e-3,
    'PATIENCE'       : 15,
    'VALVES'         : ['AV', 'MV', 'PV', 'TV'],

```

```

    'PATIENT_THRESHOLD': 0.40
}
CONFIG['WINDOW_SAMPLES'] = int(CONFIG['TARGET_SR'] * CONFIG['WINDOW_SEC'])
CONFIG['HOP_SAMPLES']     = int(CONFIG['TARGET_SR'] * CONFIG['HOP_SEC'])

```

A.3 Dataset Loading

```

df_train = pd.read_excel(CONFIG['TRAIN_CSV'])
df_train.columns = df_train.columns.str.strip()

id_col = [c for c in df_train.columns
           if 'patient' in c.lower() and 'id' in c.lower()][0]
mur_col = [c for c in df_train.columns if 'murmur' in c.lower()][0]

df_train = df_train.rename(columns={id_col: 'patient_id', mur_col:
                                     'murmur'})
df_train['patient_id'] = df_train['patient_id'].astype(str).str.strip()
df_train['label'] = (
    df_train['murmur'].str.strip().str.lower() == 'present'
).astype(int)
print(f"Total patients: {len(df_train)}")
print(df_train['murmur'].value_counts())

```

A.4 Audio Utility Functions

```

def find_wav_files(patient_id, data_root):
    pid = str(patient_id)
    patterns = [
        os.path.join(data_root, pid, f'{pid}*.wav'),
        os.path.join(data_root, f'{pid}*.wav'),
        os.path.join(data_root, pid, '*.wav'),
        os.path.join(data_root, '**', f'{pid}*.wav')
    ]
    found = []
    for p in patterns:
        found.extend(glob.glob(p, recursive=True))
    return sorted(list(set(found)))

def load_audio(filepath, target_sr):
    audio, sr = librosa.load(filepath, sr=target_sr, mono=True)
    return audio, sr

```

A.5 Signal Preprocessing Pipeline

```
def bandpass_filter(audio, sr, low=20, high=600, order=4):
    nyq = sr / 2.0
    sos = butter(order, [low/nyq, high/nyq], btype='band', output='sos')
    return sosfilt(sos, audio)

def notch_filter(audio, sr, freq=50, Q=3.0):
    nyq = sr / 2.0
    b, a = iirnotch(freq / nyq, Q)
    return filtfilt(b, a, audio)

def wavelet_denoise(audio, wavelet='db6', level=5, threshold_mode='soft'):
    coeffs = pywt.wavedec(audio, wavelet, level=level)
    sigma = np.median(np.abs(coeffs[-1])) / 0.6745
    thresh = sigma * np.sqrt(2 * np.log(len(audio)))
    denoised = [coeffs[0]] + [
        pywt.threshold(c, thresh, mode=threshold_mode) for c in coeffs[1:]
    ]
    return pywt.waverec(denoised, wavelet)[:len(audio)]

def normalize_audio(audio):
    return audio / (np.max(np.abs(audio)) + 1e-8)

def preprocess_audio(audio, sr, cfg):
    audio = bandpass_filter(audio, sr, cfg['BANDPASS_LOW'],
cfg['BANDPASS_HIGH'])
    audio = notch_filter(audio, sr, cfg['NOTCH_FREQ'], cfg['NOTCH_Q'])
    audio = wavelet_denoise(audio, cfg['WAVELET'], cfg['WAVELET_LEVEL'])
    audio = normalize_audio(audio)
    return audio
```

A.6 Feature Extraction (Windowing & Mel-Spectrogram)

```
def audio_to_windows(audio, sr, window_samples, hop_samples):
    n = len(audio)
    if n < window_samples:
        audio = np.pad(audio, (0, window_samples - n))
        n = window_samples
    windows, start = [], 0
    while start + window_samples <= n:
        windows.append(audio[start:start + window_samples])
        start += hop_samples
    return windows
```

```

def window_to_melspec(window, sr, n_mels, n_fft, hop_length,
                      f_min=20, f_max=600):
    M = librosa.feature.melspectrogram(
        y=window, sr=sr, n_mels=n_mels, n_fft=n_fft,
        hop_length=hop_length, fmin=f_min, fmax=f_max, power=2.0)
    M_db = librosa.power_to_db(M, ref=np.max)
    M_db = (M_db - M_db.min()) / (M_db.max() - M_db.min() + 1e-8)
    return M_db.astype(np.float32)

def augment_window(window, sr):
    aug = window.copy()
    target = int(sr * CONFIG['WINDOW_SEC'])
    if np.random.rand() < 0.5:
        aug = librosa.effects.time_stretch(aug, rate=np.random.uniform(0.9,
1.1))
    if np.random.rand() < 0.3:
        aug = librosa.effects.pitch_shift(aug, sr=sr,
                                         n_steps=np.random.uniform(-2, 2))
    if np.random.rand() < 0.5:
        aug += np.random.randn(len(aug)) * 0.005
    aug = aug[:target] if len(aug) > target else np.pad(aug, (0, target -
len(aug)))
    return aug

```

A.7 Dataset Builder

```

def build_dataset(df, cfg, augment=True, augment_factor=3):
    X, y, pids, missing = [], [], [], []
    for _, row in tqdm(df.iterrows(), total=len(df), desc='Building
dataset'):
        pid, label = str(row['patient_id']), int(row['label'])
        wavs = find_wav_files(pid, cfg['DATA_ROOT'])
        if not wavs:
            missing.append(pid); continue
        for fp in wavs:
            try:
                audio, sr = load_audio(fp, cfg['TARGET_SR'])
                audio = preprocess_audio(audio, sr, cfg)
                windows = audio_to_windows(audio, sr,
                                         cfg['WINDOW_SAMPLES'], cfg['HOP_SAMPLES'])
                for w in windows:
                    ms = window_to_melspec(w, sr, cfg['N_MELS'],
                                         cfg['N_FFT'], cfg['HOP_LENGTH'])

```

```

        X.append(ms); y.append(label); pids.append(pid)
        if augment and label == 1:
            for _ in range(augment_factor):
                wa = augment_window(w, sr)
                msa = window_to_melspec(wa, sr, cfg['N_MELS'],
                                         cfg['N_FFT'],
                                         cfg['HOP_LENGTH'])
            X.append(msa); y.append(label); pids.append(pid)
        except Exception as e:
            print(f"Error on {fp}: {e}")
        if missing:
            print(f"{len(missing)} patients missing audio")
    X = np.array(X)[..., np.newaxis]
    return X, np.array(y), np.array(pids)

X, y, patient_ids = build_dataset(df_train, CONFIG, augment=True,
                                  augment_factor=3)
print(f"Dataset: {X.shape} Labels: {np.bincount(y)}")

```

A.8 Train / Validation Split

```

unique_pids = df_train['patient_id'].values
unique_labels = df_train['label'].values

train_pids, val_pids = train_test_split(
    unique_pids, test_size=0.20, random_state=SEED, stratify=unique_labels)

train_mask = np.isin(patient_ids, train_pids)
val_mask = np.isin(patient_ids, val_pids)

X_tr, y_tr = X[train_mask], y[train_mask]
X_val, y_val = X[val_mask], y[val_mask]

cw = compute_class_weight('balanced', classes=np.unique(y_tr), y=y_tr)
class_weights = {i: cw[i] for i in range(len(cw))}

```

A.9 CNN-BiLSTM Model Architecture

```

def build_cnn_bilstm(input_shape, num_classes=2, l2_reg=1e-4):
    reg = regularizers.l2(l2_reg)
    inp = keras.Input(shape=input_shape, name='mel_input')

```

```

# Convolutional Block 1
x = layers.Conv2D(32, (3,3), padding='same',
kernel_regularizer=reg)(inp)
x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
x = layers.Conv2D(32, (3,3), padding='same', kernel_regularizer=reg)(x)
x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
x = layers.MaxPooling2D((2,2))(x); x = layers.Dropout(0.25)(x)

# Convolutional Block 2
x = layers.Conv2D(64, (3,3), padding='same', kernel_regularizer=reg)(x)
x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
x = layers.Conv2D(64, (3,3), padding='same', kernel_regularizer=reg)(x)
x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
x = layers.MaxPooling2D((2,2))(x); x = layers.Dropout(0.25)(x)

# Convolutional Block 3
x = layers.Conv2D(128, (3,3), padding='same', kernel_regularizer=reg)(x)
x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
x = layers.Conv2D(128, (3,3), padding='same', kernel_regularizer=reg)(x)
x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
x = layers.MaxPooling2D((2,1))(x); x = layers.Dropout(0.3)(x)

# Squeeze-Excitation Attention
se = layers.GlobalAveragePooling2D()(x)
se = layers.Dense(64, activation='relu')(se)
se = layers.Dense(128, activation='sigmoid')(se)
se = layers.Reshape((1, 1, 128))(se)
x = layers.Multiply()([x, se])

# Reshape for RNN
shape = x.shape
x = layers.Reshape((shape[2], shape[1] * shape[3]))(x)

# Bidirectional LSTM
x = layers.Bidirectional(
    layers.LSTM(128, return_sequences=True,
                dropout=0.3, recurrent_dropout=0.2))(x)
x = layers.Bidirectional(
    layers.LSTM(64, dropout=0.3, recurrent_dropout=0.2))(x)

# Classifier Head
x = layers.Dense(128, activation='relu', kernel_regularizer=reg)(x)
x = layers.Dropout(0.4)(x)

```

```

x = layers.Dense(64, activation='relu', kernel_regularizer=reg)(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(num_classes, activation='softmax', name='output')(x)

return keras.Model(inputs=inp, outputs=out, name='PCG_CNN_BiLSTM')

model = build_cnn_bilstm(X_tr.shape[1:], num_classes=2)
model.summary()

opt = keras.optimizers.AdamW(learning_rate=CONFIG['LR'], weight_decay=1e-5)
model.compile(optimizer=opt,
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

```

A.10 Model Training

```

cb_list = [
    callbacks.ModelCheckpoint(CONFIG['MODEL_PATH'],
                             monitor='val_accuracy', mode='max', save_best_only=True, verbose=1),
    callbacks.EarlyStopping(monitor='val_accuracy',
                             patience=CONFIG['PATIENCE'], mode='max',
                             restore_best_weights=True, verbose=1),
    callbacks.ReduceLROnPlateau(monitor='val_loss',
                                 factor=0.5, patience=7, min_lr=1e-6, verbose=1),
]

history = model.fit(
    X_tr, y_tr,
    validation_data=(X_val, y_val),
    epochs=CONFIG['EPOCHS'],
    batch_size=CONFIG['BATCH_SIZE'],
    class_weight=class_weights,
    callbacks=cb_list,
    shuffle=True,
    verbose=1
)

print(f'Training complete. Best model: {CONFIG["MODEL_PATH"]}')

```

A.11 Window-Level & Patient-Level Evaluation

```

# Window-level evaluation
best_model = keras.models.load_model(CONFIG['MODEL_PATH'])
y_pred_proba = best_model.predict(X_val, batch_size=64, verbose=0)

```

```

y_pred_class = np.argmax(y_pred_proba, axis=1)
print(classification_report(y_val, y_pred_class,
                            target_names=['Absent', 'Present']))
print(f'ROC-AUC: {roc_auc_score(y_val, y_pred_proba[:, 1]):.4f}')

# Patient-level prediction function
def predict_patient(patient_id, model, cfg):
    pid, all_probs = str(patient_id), []
    for fp in find_wav_files(pid, cfg['DATA_ROOT']):
        try:
            audio, sr = load_audio(fp, cfg['TARGET_SR'])
            audio = preprocess_audio(audio, sr, cfg)
            windows = audio_to_windows(audio, sr,
                                       cfg['WINDOW_SAMPLES'], cfg['HOP_SAMPLES'])
            specs = np.array([
                window_to_melspec(w, sr, cfg['N_MELS'],
                                  cfg['N_FFT'], cfg['HOP_LENGTH'])
                for w in windows
            ])[..., np.newaxis]
            preds = model.predict(specs, batch_size=32, verbose=0)
            all_probs.extend(preds[:, 1].tolist())
        except Exception as e:
            print(f"Error on {fp}: {e}")
    if not all_probs:
        return 0.5, 'Absent', 0
    pa = np.array(all_probs)
    # Weighted ensemble: 40% mean + 35% P75 + 25% max
    prob_final = (0.40 * np.mean(pa) +
                  0.35 * np.percentile(pa, 75) +
                  0.25 * np.max(pa))
    return prob_final, ('Present' if prob_final > cfg['PATIENT_THRESHOLD']
                       else 'Absent'), len(all_probs)

# Threshold optimisation
thresholds = np.arange(0.20, 0.70, 0.02)
true_binary = (df_vr['true_label'] == 'Present').astype(int)
accs = [accuracy_score(true_binary,
                       (df_vr['probability'] >= t).astype(int))
         for t in thresholds]
best_t = thresholds[np.argmax(accs)]
CONFIG['PATIENT_THRESHOLD'] = float(best_t)
print(f'Optimal threshold: {best_t:.2f} | Best Acc: {max(accs)*100:.2f}%')

```

A.12 Final Test Prediction

```

TEST_PATIENT_IDS = [
    '50009', '50012', '50014', '50015', '50017', '50018', '50023', '50026', '50027',
    '50029', '50032', '50034', '50037', '50043', '50047', '50048', '50049', '50053',
    # ... (add remaining IDs as required)
    '84937', '84949', '85002', '85004', '85024', '85030', '85031', '85036',
    '85037', '85042', '85055', '85057', '85069', '85079', '85080', '85087',
    '85108', '85110', '85119', '85132', '85133', '85135', '85145', '85165', '85169'
]

best_model = keras.models.load_model(CONFIG['MODEL_PATH'])
test_results = []
for pid in tqdm(TEST_PATIENT_IDS, desc='Predicting'):
    prob, decision, n_win = predict_patient(pid, best_model, CONFIG)
    test_results.append({'patient_id': pid,
                        'murmur': decision,
                        'confidence': f'{prob*100:.1f}%',
                        'n_windows': n_win})

df_test = pd.DataFrame(test_results)
print('\n' + '='*50)
print(' FINAL MURMUR DETECTION RESULTS')
print('='*50)
for _, row in df_test.iterrows():
    print(f"Patient {str(row['patient_id']).ljust(10)} | "
          f"Murmur: {row['murmur'].ljust(7)} | "
          f"Confidence: {row['confidence']}")
print(f"\nTotal Present: {(df_test['murmur']=='Present').sum()}")
print(f"Total Absent : {(df_test['murmur']=='Absent').sum()}")
df_test[['patient_id', 'murmur']].to_csv('test_predictions.csv', index=False)

```

A.13 Ground Truth Evaluation

```

# Populate GROUND_TRUTH dict with true labels after receiving them
GROUND_TRUTH = {
    '50009': 'Absent', '50012': 'Absent', # ... (truncated for brevity)
    '84937': 'Present', '84949': 'Present',
    '85002': 'Present', '85004': 'Present',
    # Add all ground truth labels here
}

if GROUND_TRUTH:
    df_test['true_label'] =
df_test['patient_id'].astype(str).map(GROUND_TRUTH)
df_eval = df_test.dropna(subset=['true_label'])

```

```

acc = accuracy_score(df_eval['true_label'], df_eval['murmur'])
print(f'Final Test Accuracy: {acc*100:.2f}%')
print(classification_report(df_eval['true_label'], df_eval['murmur'],
                             target_names=['Absent', 'Present']))
cm = confusion_matrix(df_eval['true_label'], df_eval['murmur'],
                      labels=['Absent', 'Present'])
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Absent', 'Present'],
            yticklabels=['Absent', 'Present'])
plt.title(f'Final Test Confusion Matrix  Acc={acc*100:.1f}%')
plt.tight_layout()
plt.savefig('final_test_confusion.png', dpi=150)
plt.show()

```